

LEVEL 17 — Missing Values in NumPy

This PDF explains missing-value functions line by line, with simple examples and outputs.

1. np.nan

np.nan means Not a Number. It is commonly used to represent a missing value in a NumPy array.

Code:

```
import numpy as np

arr = np.array([10, 20, np.nan, 40])

print(arr)
```

Output: [10. 20. nan 40.]

2. np.isnan()

Checks which values are NaN. It returns True for NaN and False for normal numbers.

Code:

```
arr = np.array([10, np.nan, 30, np.nan])

result = np.isnan(arr)

print(result)
```

Output: [False True False True]

3. np.isfinite()

Checks whether values are finite. Normal numbers are finite; NaN and infinity are not finite.

Code:

```
arr = np.array([10, np.nan, np.inf, 40])

result = np.isfinite(arr)

print(result)
```

Output: [True False False True]

4. np.isinf()

Checks whether values are positive or negative infinity.

Code:

```
arr = np.array([10, np.inf, -np.inf, 40])

result = np.isinf(arr)

print(result)
```

Output: [False True True False]

5. np.nanmean()

Calculates the mean while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanmean(arr)

print(result)
```

Output: 23.333333333333332

6. np.nanmedian()

Calculates the median while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanmedian(arr)

print(result)
```

Output: 20.0

7. np.nanstd()

Calculates standard deviation while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanstd(arr)

print(result)
```

Output: 12.47219128924647

8. np.nanvar()

Calculates variance while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanvar(arr)

print(result)
```

Output: 155.55555555555554

9. np.nansum()

Calculates the sum while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nansum(arr)

print(result)
```

Output: 70.0

10. np.nanmin()

Finds the minimum value while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanmin(arr)

print(result)
```

Output: 10.0

11. np.nanmax()

Finds the maximum value while ignoring NaN values.

Code:

```
arr = np.array([10, 20, np.nan, 40])

result = np.nanmax(arr)

print(result)
```

Output: 40.0

12. Why use nan-functions?

Normal functions such as `np.mean()`, `np.median()`, `np.std()`, `np.var()`, `np.sum()`, `np.min()`, and `np.max()` can return NaN when the array contains NaN. The nan-functions ignore NaN values and calculate using the available numeric values.

```
arr = np.array([10, 20, np.nan, 40])

print(np.mean(arr))
print(np.nanmean(arr))
print(np.sum(arr))
print(np.nansum(arr))
```

Output:

```
nan
23.333333333333332
nan
70.0
```

13. Quick Summary

- `np.nan` → represents a missing/NaN value.
- `np.isnan()` → checks for NaN.
- `np.isfinite()` → checks for finite values.
- `np.isinf()` → checks for infinity.
- `np.nanmean()` → mean, ignoring NaN.
- `np.nanmedian()` → median, ignoring NaN.
- `np.nanstd()` → standard deviation, ignoring NaN.
- `np.nanvar()` → variance, ignoring NaN.
- `np.nansum()` → sum, ignoring NaN.
- `np.nanmin()` → minimum, ignoring NaN.
- `np.nanmax()` → maximum, ignoring NaN.